

CLOUD ENGINEER ACADEMY

Complete Study Guide

Late July – December 2026

Python • AWS Cloud Practitioner • AWS Solutions Architect

Terraform • Docker • Kubernetes • CI/CD • GlamCloud AI MVP

Every module includes: learning objectives · projects with deadlines · mock interview Q&A

PHASE	DATES	PRIMARY FOCUS	MILESTONE
1 – Foundations	Jul 22–31	Python + Git + Linux	GitHub active
2 – AWS Practitioner	August	Python Intermediate + AWS CLF	Pass AWS Cloud Practitioner
3 – AWS Architect	September	AWS SAA + GlamCloud Start Pass	SAA by Sep 30 + Begin GlamCloud AI
4 – IaC	October	Terraform + Git/GitHub	Resume + LinkedIn + Apply
5 – Containers	November	Docker + Linux + CI/CD	10+ applications/week
6 – Orchestration	December	Kubernetes + Security + Jobs	GlamCloud MVP + Hired

MODULE 1: Python Foundations + Git + Linux

Late July 2026 | July 22 – July 31 | 10 Days

Overview

You're launching the academy today. This first sprint is about building momentum and good habits. You will set up your development environment, write your first Python programs, learn the Linux terminal, and make your GitHub profile active. Every day counts — commit code every single day.

Daily Schedule (3–4 hrs)

TIME	ACTIVITY
8:00–9:30 AM	Python study (new concept + notes)
9:30–9:45 AM	Break
9:45–11:15 AM	Python coding practice / mini lab
11:15–11:45 AM	Linux terminal practice OR Git/GitHub
Evening (optional)	30 min review, GitHub commit, journal entry

Topics to Cover

- Python: variables, data types (int, float, str, bool), print statements, input()
- Python: if/elif/else, comparison operators, logical operators
- Python: for loops, while loops, range()
- Python: functions — def, parameters, return values
- Python: lists — indexing, slicing, append, remove, iteration
- Python: dictionaries — key/value pairs, get, update, iteration
- Linux: ls, cd, mkdir, rm, cp, mv, cat, echo, pwd, man
- Linux: file permissions (chmod), nano/vim basics
- Git: init, add, commit, push, pull, clone, status, log
- GitHub: create repo, write a README.md, push from terminal

Projects + Deadlines

#	PROJECT	DEADLINE	DELIVERABLE
1	BMI Calculator	July 26	Python script that takes height/weight, calculates BMI, prints category
2	Password Generator	July 31	Python script using random module; lets user choose length and complexity

■ **DEADLINE: Both projects pushed to GitHub with README by July 31**

What You Should Know by the End of Module 1

- ✓ Write a Python script from scratch using variables, loops, functions, and lists
- ✓ Explain the difference between a list and a dictionary
- ✓ Navigate the Linux file system using the terminal without looking anything up
- ✓ Create a GitHub repo, add files, and push commits from the command line
- ✓ Explain what Git is and why version control matters

Mock Interview — Module 1

Q: What is the difference between a list and a dictionary in Python?

A: A list is an ordered collection accessed by index (`list[0]`). A dictionary is an unordered collection of key-value pairs accessed by key (`dict['name']`). Use a list when order matters; use a dict when you need to label your data.

Q: What does the 'def' keyword do in Python?

A: It defines a function. You give the function a name, optional parameters, and a body. Functions allow you to write reusable, organized code rather than repeating the same logic multiple times.

Q: What is Git and why do engineers use it?

A: Git is a version control system that tracks changes to your code over time. Engineers use it so they can collaborate without overwriting each other's work, revert to previous versions if something breaks, and maintain a full history of every change ever made.

Q: What is the difference between 'git add' and 'git commit'?

A: 'git add' stages your changes — it selects what will be included in the next snapshot. 'git commit' takes that staged snapshot and saves it permanently to your local repository history with a message describing what changed.

Q: What does 'chmod 755' mean in Linux?

A: chmod changes file permissions. 755 means the owner can read, write, and execute (7), while the group and others can only read and execute (5). This is a common permission for executable scripts.

MODULE 2: Python Intermediate + AWS Cloud Practitioner

August 2026 | August 1 – August 31 | ~26 Study Days

Overview

This is your most important month. You have two parallel tracks: deepening your Python skills to intermediate level, and studying for the AWS Cloud Practitioner certification. The cert exam is your first major milestone and signals to employers that you are serious about cloud. Study Python mornings, AWS afternoons. Build projects that combine both.

Daily Schedule (3–4 hrs)

TIME	ACTIVITY
8:00–9:30 AM	Python intermediate lesson (see topics below)
9:30–9:45 AM	Break
9:45–11:15 AM	Python practice project or coding exercise
11:15–11:45 AM	AWS Cloud Practitioner study (Stephane Maarek or Andrew Brown)
Evening (optional)	AWS flashcards, practice questions, GitHub commit

Python Topics to Cover

- File I/O — open(), read(), write(), with statement
- Exception handling — try/except/finally, custom exceptions
- Working with JSON — json.loads(), json.dumps(), reading JSON files
- List comprehensions, dictionary comprehensions
- Modules and imports — os, sys, math, random, datetime
- APIs — using the requests library, parsing JSON responses
- Object-Oriented Programming basics — classes, __init__, methods, self

AWS Cloud Practitioner Topics

- Cloud concepts: IaaS vs PaaS vs SaaS, shared responsibility model, cloud benefits
- IAM: users, groups, roles, policies, MFA, least privilege principle
- EC2: instance types, AMIs, security groups, key pairs, pricing models
- S3: buckets, objects, storage classes, versioning, static website hosting
- VPC: subnets, route tables, internet gateways, NAT gateways
- RDS: managed relational databases, Multi-AZ, read replicas
- Lambda: serverless compute, triggers, event-driven architecture
- CloudWatch: metrics, alarms, logs, dashboards
- Billing: Cost Explorer, Budgets, Free Tier, pricing calculator
- Global infrastructure: regions, availability zones, edge locations

Projects + Deadlines

#	PROJECT	DEADLINE	DELIVERABLE
---	---------	----------	-------------

3	Expense Tracker	Aug 10	Python CLI app that saves expenses to a JSON file; calculates tot
4	Contact Book	Aug 17	Python CLI CRUD app stored in JSON; add, search, update, dele
5	Weather API App	Aug 24	Calls OpenWeatherMap API; displays forecast for any city; handle
6	S3 Static Website	Aug 28	Host a simple HTML/CSS site on S3 with public access and static
7	EC2 Web Server	Aug 31	Launch EC2 instance, install nginx/apache, host a webpage, confi

■ **DEADLINE: AWS Cloud Practitioner exam: TARGET August 31**

What You Should Know by the End of Module 2

- ✓ Read/write files and parse JSON data in Python without referencing documentation
- ✓ Write a Python script that calls a REST API and handles errors
- ✓ Explain the AWS shared responsibility model and the six pillars of the Well-Architected Framework
- ✓ Describe what IAM, EC2, S3, VPC, RDS, Lambda, and CloudWatch do — and when to use each
- ✓ Pass the AWS Cloud Practitioner exam (CLF-C02)
- ✓ Have 5 projects on GitHub, each with a professional README

Mock Interview — Module 2

Q: Explain the AWS shared responsibility model.

A: AWS is responsible for the security OF the cloud — the physical hardware, data centers, networking, and hypervisor. Customers are responsible for security IN the cloud — their data, operating systems, network configurations, IAM policies, and application code.

Q: What is IAM and why is the principle of least privilege important?

A: IAM (Identity and Access Management) controls who can access AWS resources and what they can do. Least privilege means granting only the permissions someone needs to do their job and nothing more. This limits the blast radius if credentials are ever compromised.

Q: What is the difference between S3 Standard and S3 Glacier?

A: S3 Standard is for frequently accessed data — it has low latency and high throughput. S3 Glacier is for archival storage — it is much cheaper but retrieval takes minutes to hours. You choose based on how often you need the data and your cost constraints.

Q: What is a VPC and why would you use one?

A: A Virtual Private Cloud is a logically isolated section of AWS where you can define your own network, subnets, route tables, and security rules. You use one to control what resources can talk to each other and what is exposed to the internet.

Q: Why would you use Lambda instead of EC2?

A: Lambda is serverless — you only pay for the milliseconds your code runs and AWS manages all the infrastructure. EC2 is a virtual machine you provision and manage yourself. Lambda is ideal for event-driven workloads, short-lived tasks, and APIs where you don't want to manage servers.

Q: How do you handle an exception in Python? Give an example.

A: You use a try/except block. For example: 'try: result = 10 / 0' and 'except ZeroDivisionError: print("Cannot divide by zero")'. You can also add 'finally' to run cleanup code regardless of whether an error occurred.

MODULE 3: AWS Solutions Architect Associate + GlamCloud Start

September 2026 | September 1 – September 30 | ~26 Study Days

Overview

SAA-C03 is a heavyweight cert. Employers take it seriously because it proves you can design real cloud architectures. Study deeply — don't memorize, understand. This month you also kick off GlamCloud AI, your flagship portfolio project. Begin building the repository structure and document your first Architecture Decision Records (ADRs). Take at least one practice exam every week.

AWS SAA Topics to Cover

- EC2 advanced: placement groups, hibernate, instance metadata, user data scripts
- VPC deep dive: public/private subnets, NAT Gateway, VPC peering, VPC endpoints, Flow Logs
- S3 advanced: lifecycle policies, cross-region replication, presigned URLs, S3 Select
- Route 53: routing policies — simple, weighted, latency, failover, geolocation, geoproximity
- Load Balancers: ALB vs NLB, target groups, sticky sessions, SSL/TLS termination
- Auto Scaling: launch templates, scaling policies, cooldown, health checks
- RDS: Multi-AZ vs read replicas, encryption, automated backups, parameter groups
- DynamoDB: partition key, sort key, GSIs, LSIs, capacity modes, DAX
- Lambda: concurrency, layers, environment variables, VPC integration, destinations
- CloudFront: distributions, origins, caching behaviors, signed URLs, OAI/OAC
- IAM advanced: roles for services, resource-based policies, SCP, permission boundaries
- SQS, SNS: decoupled architecture, fan-out pattern, FIFO queues
- ECS, ECR, Fargate: container orchestration basics
- CloudFormation: stacks, templates, drift detection

Projects + Deadlines

#	PROJECT	DEADLINE	DELIVERABLE
8	Serverless Image Uploader	Sep 12	API Gateway + Lambda + S3. User uploads image via API; Lambda uploads to S3
9	Cloud Resume Challenge	Sep 21	S3 + CloudFront + Route 53 + DynamoDB + Lambda + API Gateway
	GlamCloud GlamCloud	Sep 30	Create repo with professional structure: ADRs, architecture diagrams

■ **DEADLINE: AWS Solutions Architect Associate exam: TARGET September 30, 2026**

■ **DEADLINE: AWS SAA weekly practice exams every Saturday | Begin GlamCloud repo by Sep 30**

What You Should Know by the End of Module 3

- ✓ Design a highly available, fault-tolerant architecture on AWS (multi-AZ, load balancing, auto scaling)
- ✓ Choose the right database service for a given scenario (RDS vs DynamoDB vs ElastiCache)
- ✓ Explain CloudFront's role in performance and how it integrates with S3
- ✓ Describe how Route 53 routing policies work and when to use each
- ✓ Build a serverless API using Lambda, API Gateway, and S3
- ✓ Have GlamCloud AI repo initialized with professional documentation structure on GitHub

Mock Interview — Module 3

Q: What is the difference between an ALB and an NLB?

A: An Application Load Balancer (ALB) operates at Layer 7 (HTTP/HTTPS) and can route based on URL path, hostname, or headers. A Network Load Balancer (NLB) operates at Layer 4 (TCP/UDP) and is designed for ultra-low latency and millions of requests per second. Use ALB for web apps; NLB for gaming, IoT, or real-time streaming.

Q: What is the difference between Multi-AZ and a read replica in RDS?

A: Multi-AZ creates a synchronous standby in a different AZ for high availability and automatic failover — it is not for scaling reads. A read replica is an asynchronous copy used to scale read traffic. Multi-AZ is for disaster recovery; read replicas are for performance.

Q: Explain the Cloud Resume Challenge and what services it uses.

A: It is a project where you host your resume as a website on S3 with CloudFront for HTTPS delivery, a visitor counter stored in DynamoDB, a Lambda function that updates the counter via API Gateway, and a GitHub Actions pipeline for automated deployment. It demonstrates end-to-end cloud architecture skills.

Q: What is an Architecture Decision Record (ADR)?

A: An ADR documents why a specific technical decision was made — the context, the options considered, the decision taken, and the consequences. They help teams understand past decisions and avoid repeating the same discussions. They are a sign of engineering maturity.

Q: When would you use SQS vs SNS?

A: SQS is a message queue where one consumer processes each message — useful for task queues and decoupling services. SNS is a pub/sub system that fans messages out to multiple subscribers simultaneously. A common pattern is to combine them: SNS sends a notification, which triggers multiple SQS queues for different consumers.

MODULE 4: Terraform + Git/GitHub + Internship Preparation

October 2026 | October 1 – October 31 | ~26 Study Days

Overview

Infrastructure as Code separates junior engineers from everyone else. This month you will stop clicking in the AWS console and start writing code to provision infrastructure. You will also go deep on professional Git workflows — branching strategies, pull requests, commit conventions. Your resume and LinkedIn go live this month, and you start applying to internships.

Terraform Topics to Cover

- HCL syntax: providers, resources, variables, outputs, locals, data sources
- State: terraform.tfstate, remote state with S3 + DynamoDB locking
- Commands: init, plan, apply, destroy, fmt, validate, import, state
- Modules: creating reusable modules, calling modules from root config
- Workspaces: managing multiple environments (dev, staging, prod)
- Meta-arguments: count, for_each, depends_on, lifecycle
- AWS provider: provisioning EC2, VPC, S3, IAM roles, security groups

Git/GitHub Advanced Topics

- Branching strategies: Git Flow, trunk-based development, feature branches
- Pull requests: writing good PR descriptions, code review etiquette
- Conventional commits: feat:, fix:, docs:, chore:, breaking changes
- GitHub: branch protection rules, required reviewers, status checks
- .gitignore: what to ignore (secrets, .tfstate files, venv, __pycache__)
- Git rebase vs merge: when to use each

Projects + Deadlines

#	PROJECT	DEADLINE	DELIVERABLE
10	Terraform Infrastructure	Oct 14	Provision EC2 + VPC + security groups + S3 bucket using Terraform
11	VPC Project	Oct 24	Build a full VPC with public/private subnets, NAT gateway, bastion
Resumé and LinkedIn	Completed	Oct 31	Professional resume listing projects, certifications, and skills; LinkedIn

■ **DEADLINE: Resume live and internship applications begin: October 31**

What You Should Know by the End of Module 4

- ✓ Write Terraform code to provision any basic AWS infrastructure from scratch
- ✓ Explain what terraform plan does and why you always run it before apply
- ✓ Manage Terraform state safely using remote backend (S3 + DynamoDB)
- ✓ Create and use a reusable Terraform module
- ✓ Follow professional Git workflow: feature branch → pull request → merge
- ✓ Have a polished resume and LinkedIn profile ready for employers

Mock Interview — Module 4

Q: What is Infrastructure as Code and why does it matter?

A: IaC means managing and provisioning infrastructure through code files rather than manual processes. It matters because it makes infrastructure reproducible, version-controlled, and auditable. You can spin up identical environments in minutes, catch configuration drift, and collaborate with teammates using the same Git workflows you use for application code.

Q: What does 'terraform plan' do?

A: It shows you exactly what Terraform will create, modify, or destroy before it makes any changes. It is a dry run that compares your configuration against the current state file. You should always review the plan output before running 'terraform apply' — especially in production.

Q: What is Terraform state and why is it important to store it remotely?

A: The state file (terraform.tfstate) is Terraform's record of what infrastructure it manages. It maps your HCL config to real cloud resources. You store it remotely (S3 + DynamoDB) so your team can collaborate without state conflicts, so it is backed up, and so state locking prevents two people from running apply at the same time.

Q: What is the difference between 'count' and 'for_each' in Terraform?

A: 'count' creates N identical resources using a number. 'for_each' iterates over a map or set, creating one resource per element with a unique key. for_each is generally preferred because removing an element from the middle of a count list causes Terraform to re-number everything, potentially destroying and recreating resources.

Q: Walk me through a project on your GitHub. What problem does it solve?

A: Pick your strongest project. Structure your answer: 'I built X to solve Y problem. I used [technologies]. The architecture is [brief description]. The hardest challenge was [honest answer]. Here is what I would do differently: [shows growth mindset].'

Polish MODULE 5: Docker + Linux + CI/CD + Portfolio

November 2026 | November 1 – November 30 | ~25 Study Days

Overview

Containerization is now a baseline skill for cloud engineers. This month you will learn Docker deeply, set up automated CI/CD with GitHub Actions, and continue daily Linux practice. You will also be applying to 10+ internships per week — the portfolio you built is your proof of work. Practice interview questions twice a week on Saturdays and Wednesdays.

Docker Topics to Cover

- Images vs containers: layers, base images, image registry (Docker Hub, ECR)
- Dockerfile: FROM, RUN, COPY, WORKDIR, EXPOSE, CMD, ENTRYPOINT, ENV, ARG
- Commands: docker build, run, ps, stop, rm, exec, logs, inspect, pull, push
- Volumes: bind mounts vs named volumes; persisting data
- Networking: bridge, host, none, container-to-container communication
- Docker Compose: multi-container apps, services, depends_on, environment files
- Best practices: .dockerignore, multi-stage builds, non-root users, minimal images

GitHub Actions / CI/CD Topics

- Workflow syntax: .github/workflows/*.yml, on triggers, jobs, steps, uses
- Actions marketplace: actions/checkout, actions/setup-python, aws-actions
- Secrets: storing AWS credentials, environment variables in GitHub Settings
- CI pipeline: lint → test → build → push to ECR
- CD pipeline: deploy to EC2 or Lambda on merge to main
- Status badges and branch protection rules tied to workflow status

Projects + Deadlines

#	PROJECT	DEADLINE	DELIVERABLE
11	Dockerized FastAPI App	Nov 14	FastAPI REST API containerized with Docker; pushed to Docker Hub
12	Monitoring Dashboard	Nov 21	CloudWatch dashboard + Lambda alarm + SNS notification for a service
14	CI/CD Pipeline	Nov 30	GitHub Actions pipeline that lints, tests, builds Docker image, and deploys to EC2 or Lambda

■ **DEADLINE: Apply to 10+ internships/jobs every week throughout November**

What You Should Know by the End of Module 5

- ✓ Write a Dockerfile from scratch and explain every instruction
- ✓ Run a multi-container application using Docker Compose
- ✓ Explain what a CI/CD pipeline is and build one with GitHub Actions
- ✓ Push a Docker image to ECR and describe the workflow
- ✓ Use Linux commands fluently: grep, awk, sed, ps, top, netstat, curl, ssh
- ✓ Be actively applying to internships with a strong portfolio behind every application

Mock Interview — Module 5

Q: What is the difference between a Docker image and a Docker container?

A: An image is a read-only blueprint — like a class in programming. A container is a running instance of that image — like an object. You can run multiple containers from the same image, each isolated from one another.

Q: What is a multi-stage Docker build and why would you use it?

A: A multi-stage build uses multiple FROM statements in a single Dockerfile. You build your application in a heavy base image (with compilers and dev tools), then copy only the compiled output into a minimal runtime image. The final image is much smaller and has a smaller attack surface.

Q: Explain what CI/CD means and why companies use it.

A: CI (Continuous Integration) means developers merge code frequently and automated tests run on every commit to catch bugs early. CD (Continuous Delivery/Deployment) means that passing code is automatically deployed to staging or production. Together they reduce the time between writing code and getting it in front of users, and they reduce human error from manual deployments.

Q: How do you store secrets in a GitHub Actions workflow?

A: You store them as encrypted secrets in the repository or organization settings under Settings > Secrets. In the workflow file you reference them as `${{ secrets.MY_SECRET }}`. Never hardcode credentials in your workflow files or commit them to your repo.

Q: Tell me about a time you debugged a problem. What was your process?

A: Use the STAR method: Situation (what were you building), Task (what went wrong), Action (how you debugged — logs, isolation, documentation), Result (what you fixed and what you learned). Be specific and honest.

MODULE 6: Kubernetes + Security + GlamCloud MVP + Job Search

December 2026 | December 1 – December 31 | Final Sprint

Overview

This is your culmination month. You will learn Kubernetes fundamentals, complete your GlamCloud AI MVP, finish the CI/CD pipeline, and aggressively pursue interviews. By December 31 you should have 10+ GitHub projects, AWS Cloud Practitioner certified, a polished resume and LinkedIn, and active applications to internships and junior cloud roles. Keep applying 10–15 positions per week and practice interviews every weekend.

Kubernetes Topics to Cover

- Architecture: control plane (API server, etcd, scheduler, controller manager) vs worker nodes
- Core objects: Pods, ReplicaSets, Deployments, Services, Namespaces
- Networking: ClusterIP vs NodePort vs LoadBalancer, Ingress controllers
- Configuration: ConfigMaps, Secrets, environment variables in pods
- Storage: PersistentVolumes, PersistentVolumeClaims, StorageClasses
- Health checks: liveness probes, readiness probes, startup probes
- kubectl: get, describe, apply, delete, logs, exec, port-forward, rollout
- EKS: managed Kubernetes on AWS, node groups, IAM roles for service accounts

Cloud Security Topics

- IAM best practices: no root access, MFA on all accounts, role-based access
- Encryption: at rest (KMS, S3 SSE) vs in transit (TLS/SSL, ACM)
- Secrets management: AWS Secrets Manager vs Parameter Store
- Security groups vs NACLs: stateful vs stateless, evaluation order
- AWS Shield, WAF, GuardDuty, Security Hub: what each protects against
- Vulnerability scanning: ECR image scanning, Trivy for container images

Projects + Deadlines

#	PROJECT	DEADLINE	DELIVERABLE
13	Kubernetes Deployment	Dec 14	Deploy Dockerized app to EKS or Minikube; write Deployment + S
15	GlamCloud AI MVP	Dec 28	Complete flagship project: full architecture, Terraform IaC, CI/CD,
Final	Portfolio Review	Dec 31	All 15 projects on GitHub with READMEs; portfolio site live; resum

■ **DEADLINE: GlamCloud AI MVP complete by December 28 | Portfolio complete by December 31**

What You Should Know by the End of Module 6 (End of Bootcamp)

- ✓ Deploy and manage containerized applications on Kubernetes
- ✓ Write Kubernetes manifests (Deployment, Service, ConfigMap, Secret, Ingress)
- ✓ Explain the AWS shared responsibility model as it applies to security
- ✓ Describe how to secure secrets in both application code and Kubernetes
- ✓ Present GlamCloud AI confidently in 2–3 minutes, explaining architecture decisions

- ✓ Answer technical interview questions across Python, AWS, Terraform, Docker, and Kubernetes
- ✓ Be AWS Cloud Practitioner certified with SAA in strong progress
- ✓ Have an active, professional GitHub with 15 projects and consistent commit history

Mock Interview — Module 6 (Full Technical Interview Simulation)

Q: Walk me through your GlamCloud AI project — what does it do and how is it architected?

A: 'GlamCloud AI is my flagship portfolio project — an AI-powered platform for the beauty industry hosted on AWS. The backend is built with [Python/FastAPI], containerized with Docker, deployed using Kubernetes on EKS. Infrastructure is provisioned with Terraform, deployments are automated via GitHub Actions CI/CD. I documented all major decisions in Architecture Decision Records and the project includes a full security review.' Tailor this to your actual build.

Q: What is the difference between a Deployment and a ReplicaSet in Kubernetes?

A: A ReplicaSet ensures a specified number of pod replicas are running at any time. A Deployment is a higher-level abstraction that manages ReplicaSets and provides rolling updates, rollback capabilities, and declarative updates. In practice you almost always use Deployments, not ReplicaSets directly.

Q: What is the difference between a ConfigMap and a Secret in Kubernetes?

A: Both store configuration data, but Secrets are for sensitive data (passwords, tokens, keys) and are base64-encoded and can be encrypted at rest in etcd. ConfigMaps are for non-sensitive configuration like environment settings. Neither should store credentials in a production system without additional encryption — use AWS Secrets Manager instead.

Q: Why is least privilege important and how do you enforce it on AWS?

A: Least privilege limits the damage if a credential is compromised — the attacker can only do what that credential was allowed to do. On AWS you enforce it with IAM: write narrow policies granting only specific actions on specific resources, use roles instead of long-lived access keys, enable MFA, and regularly audit with IAM Access Analyzer and Trusted Advisor.

Q: Why do you want to work in cloud engineering?

A: Be authentic. Something like: 'I love that cloud engineering sits at the intersection of software, infrastructure, and scale. I spent this year building [specific projects], getting AWS certified, and learning how production systems actually work. Cloud is where I want to build my career and I am ready to contribute from day one.'

Q: Where do you see yourself in two years?

A: 'I want to be a confident cloud or DevOps engineer who designs and maintains real production systems. My short-term goal is to land this internship, absorb everything I can, and earn my AWS Solutions Architect certification. Within two years I want to be working on infrastructure that serves real users at scale.'

Academy Rules + End-of-Year Checklist

Keep this page visible. Return to it every Sunday.

Daily Non-Negotiables

- Study Mon–Sat. Sunday = rest + review + GitHub organization.
- Push at least one commit to GitHub every study day.
- Complete every lab before moving to the next topic.
- Write a README for every project before moving on.
- Spend 30 minutes on AWS flashcards or practice questions (Aug–Sep).
- Log one engineering journal entry per week.

End-of-Year Achievement Checklist

STATUS	ACHIEVEMENT
☐	AWS Cloud Practitioner certified (target: Aug 31)
☐	AWS Solutions Architect Associate — in progress or complete
☐	Python: can write intermediate scripts, APIs, and OOP code
☐	Terraform: can provision AWS infrastructure from code
☐	Docker: can containerize any application and push to a registry
☐	Kubernetes: can deploy and manage pods, services, and deployments
☐	15 GitHub projects complete, each with a README
☐	GlamCloud AI MVP deployed and documented
☐	Professional resume — updated monthly since October
☐	LinkedIn profile live with projects and certifications
☐	Applying to 10–15 internships/jobs weekly since November
☐	Mock interview practice every Saturday throughout bootcamp
☐	Weekly engineering journal maintained

**You built this. Every commit, every project, every mock interview answer — it compounds.
Stay consistent, stay curious, and keep going.**